

Behavior-cloning the research orchestrator

What I built this week, with the numbers from the runs

Srikar Vadlamani

PILOT stage 1 ▪ Deep Research Bench

September 23, 2026

The one-sentence version

This week

I collected randomized orchestrator trajectories, cloned the best ones into a Qwen2.5-7B LoRA, and ran that policy against a frozen rule on the validation queries.

- ① **Randomized rollouts.** 300 train trajectories. 167 actually had evidence.
- ② **Behavior cloning.** 113 state-action rows from the top 25% by reward.
- ③ **LoRA.** 3 epochs on an A100. Dev parses at 100%. No invented evidence ids.
- ④ **Val**, $n = 8$. RACE is a tie (+0.003). FACT support is higher for the fixed rule (0.72 vs. 0.64). Learned costs about twice the tokens and the time.

Test queries 51–55 were not used to pick anything.

What the orchestrator decides

After every research round the policy reads the evidence pool and writes exactly three lines.

Action (u, m, w)

- KEEP — which pages stay in context
- ALLOC — how the next search budget is split
- DECISION — CONTINUE or TERMINATE

Same text online and offline

- The fork and the BC builder share one serializer.
- The prompt the model sees at eval is the prompt it was trained on.
- A bad answer does not crash the run: it keep-alls and continues, and the log says `parse_failure`.

Source: `gpt_researcher/orchestration/serialize.py`. Eight features per page: relevance, redundancy, coverage, length, novelty, depth, round, new-or-old.

(u, m, w) in plain language

One research round ends. The policy must return three things. The letters are the paper's

names for them.	Letter	In the log	Meaning	If u is
	u	DECISION	Stop and write the report, or search again.	
	m	KEEP	The set of page ids that stay in the working context. Everything else is dropped.	
	w	ALLOC	How the next round's searches are split across the open branches. Weights sum to 1.	

TERMINATE, w is ignored: there is no next search. The report is written from the pages in m .

What one page looks like

Query 88, round 1. This page was **kept**.

```
id:      0609ac4b5505
url:      deadline.com/.../one-hundred-years-of-solitude-...
from:     "Garcia Marquez estate Netflix negotiation ..."
depth: 1      round: 1      words: 1222
```

The orchestrator does not read the whole article. It sees a short line: the id, eight numbers, and a snippet.

ρ	nov	red	cov	tok	depth	age	src
relevance	new?	duplicate?	covers a sub-question?	length	tree level	how old	.gov/.edu

Real page: Deadline, 6 March 2019, "Netflix to adapt ...One Hundred Years of Solitude." File:
88-s0/gpt_researcher/88.json.

Every variable, and who uses it

Three different objects. Do not mix them.

Object	Variables
State the LoRA reads	Root question, sub-questions, round, tree depth, tokens used, context budget, and for each page: id, the 8 features below, new/retained, source domain, 160-character snippet. For each open branch: id, coverage gap, subquery.
Action it must emit	u terminate, m kept ids, w branch weights.
Knobs that <i>made</i> the training actions	The 11 PARAM_SPACE draws (keep ratio, weights, stop rule, ...). Not shown to the LoRA.
Reward that <i>picked</i> which actions to clone	Q (RACE), Q floored at 0.44, token cost, latency cost, coverage Φ .

Embeddings (text-embedding-3-small) are only used to *compute* ρ , novelty, redundancy, coverage, and Φ . The prompt does not contain the vectors.

The 8 numbers on each page

Order is fixed: rho nov red cov tok depth age src. From `features.py`.

Name	Symbol	Definition
Relevance	ρ	Cosine of this page with the root question.
Novelty	nov	1 – cosine with the most similar page already kept.
Redundancy	red	Fraction of other pages with cosine > 0.85 .
Coverage	cov	Fraction of sub-questions this page matches (cosine > 0.50).
Length	tok	Estimated tokens, $\lfloor \text{chars}/4 \rfloor$.
Depth	depth	Tree level that retrieved it.
Age	age	Rounds since it was retrieved.
Source	src	2 = .gov/.edu, 1 = .org or Wikipedia, 0 = anything else.

Two constants, not learned:

$\tau = 0.85$ (near-duplicate) and $\tau_c = 0.50$ (covers a sub-question). Branch gap = 1 – best cosine between a kept page and that branch's subquery.

What (u, m, w) looked like on a round that continues

Learned policy, query 64, round 1. Pool of 39 pages. Four open branches.

KEEP: <36 ids>

ALLOC: A=0.11 B=0.05 C=0.41 D=0.43

DECISION: CONTINUE

- m : 36 kept, 3 dropped.
- w : the next search is split. C and D get about 40% each. B is almost off.
- u : CONTINUE. Round 2 only searches the branches w funded.

parse_failure=false. File: runs/eval_val_learned/64-s0/gpt_researcher/64.json. Query 88 is the wrong picture for w : its only round set one branch to 1.00 and then terminated, so the weights were never used.

What a trajectory is

A trajectory is the whole run of one question, not one decision.

- ① Planner writes sub-questions.
- ② For each round: search, scrape, then one (u, m, w) .
- ③ If u is continue, the next round searches only the branches w funded, and the pool is whatever m kept plus the new pages.
- ④ If u is terminate, the writer turns the kept pages into the report.

Query 88's learned trajectory has **one** round, then a report. A deeper one (learned 64) has **six** rounds: six keep-sets, six allocations, and a terminate on the last.

What a trajectory looks like on disk

runs/eval_val_learned/88-s0/

trajectories/88.json	steps: search, prune, report
gpt_researcher/88.json	rounds[], decision.meta, pages
reports/88.md	the article that was scored
metrics/summary.json	pages kept, tokens, RACE

Two views of the same run.

- Harness step 1 is {"type": "prune", "rationale": "kept 37, pruned 10"}.
- Fork round 1 stores the three-line action and parse_failure.

The train corpus is the same shape, under runs/rollouts_drb_d3b4/57-s0/ and so on.
Behavior cloning reads the fork file, not the markdown report.

The frozen rule, exactly

Val fixed-policy command pins four of the eleven knobs. The other seven are still drawn once per question from seed 0.

```
GR_RANDOM_PARAMS = {  
  "keep_ratio": 0.8,  
  "min_rounds": 2,  
  "p_terminate": 1.0,  
  "temperature": 0.02  
}
```

- **keep_ratio 0.8.** On a pool of 47, keep $\lceil 0.8 \times 47 \rceil = 38$ pages. Not “keep 80% of the internet.”
- **temperature 0.02.** Almost no ranking noise. The order is the weighted score.
- **min_rounds 2.** Cannot stop after round 1, even if coverage is already high.
- **p_terminate 1.0.** As soon as round 2 is allowed, the coin flip is heads with probability 1. It always stops there.

So the fixed-policy means, on the 2 rounds, keep about 80% each time, then write the report

Splits, so we do not cheat

Deep Research Bench, English, seed 17. Depth 3, breadth 4. Scraper: BeautifulSoup. Budget:

150k tokens / 900s, not enforced.	Split	n	Role
	Train	30	Randomized corpus, then BC
	Val	10	Fixed rule vs. learned. Not used to train.
	Test	10	Held out. Table 4 pins are 51–55.

Val ids: 59, 64, 65, 69, 84, 85, 88, 94, 95, 100.

File: `data/benchmarks/deep_research_bench/splits.json`.

1. Why the rollouts are randomized

The Table-4 baselines (none, topk, ...) are deterministic given the state. Every trajectory of topk keeps the same kind of page. There is no action diversity, so there is nothing to clone. RandomizedPolicy (orchestration/randomized.py) does the opposite:

- Sample **11 orchestration parameters once** per query.
- Reuse that same vector on every later round of that query.
- The decision is still a heuristic (rank, allocate, stop). The randomness is in *how aggressive* the heuristic is.

Ten seeds on one question therefore span “keep almost everything” through “keep 15%”, and “stop at round 1” through “keep going”.

How one rollout is launched

`adr rollouts` does not run 300 queries inside one process. `gpt-researcher`'s token counters are process-global, so each rollout is its own Python process.

- 1 Plan every pair (query, seed) from the train split. Seeds are $0 \dots 9$.
- 2 `run_id` is `<qid>-s<seed>`, for example `57-s3`.
- 3 Skip the pair if `trajectories/<qid>.json` already has a report and no error. The corpus is resumable.
- 4 Subprocess: `adr run` with `GR_ORCHESTRATOR=random` and `GR_ORCH_SEED=<seed>`.
- 5 After the report, RACE is judged one rollout at a time. Two judges at once would overwrite the same file.

Driver: `src/adr/rollouts/driver.py`. Config: `configs/rollouts_drb_random_d3b4.yaml`.

The frozen agent every seed shared

Nothing except the orchestrator changed across the 300 runs.

Dataset	Deep Research Bench, train split, 30 queries	Agent
Tree	depth 3, breadth 4, 5 search hits per subquery	
Scraper	HTTP + BeautifulSoup (no Chrome)	
Search	Tavily, key file env/tavily.csv	
Writer / planner	Azure gpt-5-mini	
Embeddings	text-embedding-3-small	
Budget written down, not enforced	150k tokens, 900 seconds	

file: configs/agents/gpt_researcher_d3b4.yaml. Output root: runs/rollouts_drb_d3b4/.

The 11 parameters, sampled once

From PARAM_SPACE in randomized.py. Log-uniform means the sampler draws uniformly in log space, so small and large values are both reachable.

Name	Draw	What it does
keep_ratio	Unif(0.15, 1)	Fraction of the pool to keep
budget_frac	Unif(0.3, 1)	Token cap on the kept set
temperature	LogUnif(0.02, 0.5)	How much Gumbel noise shakes the ranking
w_nov, w_red, w_cov	Unif(0, 1)	Feature weights
w_len	Unif(0, 0.5)	Penalty on long pages
alloc_concentration	LogUnif(0.5, 20)	Peaked vs. flat branch split
min_rounds	{1, 2, 3}	Cannot stop before this round
p_terminate	Unif(0, 0.25)	Coin-flip stop, after min_rounds
phi_gain_stop	Unif(0, 0.05)	Stop if coverage gain is tiny

What the code does with those parameters

Keep. Score each page

$$\rho + w_{\text{nov}}\text{nov} - w_{\text{red}}\text{red} + w_{\text{cov}}\text{cov} - w_{\text{len}}\widehat{\text{tok}}$$

add Gumbel noise scaled by temperature, and keep the top $\lceil \text{keep_ratio} \cdot n \rceil$ that still fit under the token cap. **Allocate.** Dirichlet whose mean follows each open branch's coverage

gap. High `alloc_concentration` puts almost all of the next search on the biggest gap.

Stop. Only after `min_rounds`. Then stop on a `Bernoulli(p_terminate)` draw, or if the coverage gain fell below `phi_gain_stop`. The seed is `sha256(GR_ORCH_SEED | question)`, so seed 3 on query 57 is the same trajectory if you rerun it.

What gets written, and why BC can replay it

Each round's `decision.meta` stores the parameters, the eight features of every page, the frontier gaps, Φ before and after, and the seed. The offline builder reads that meta and calls the **same** `serialize_state` the online policy uses. The training prompt is not a reconstruction from memory. It is the string the policy would have been shown. On disk, one rollout is a folder:

- `57-s0/trajectories/57.json` — harness steps and the report
- `57-s0/gpt_researcher/57.json` — the fork's rounds and meta
- `57-s0/metrics/summary.json` — tokens, reads, retained pages
- `logs/57-s0.log` — the subprocess log
- `manifest.jsonl` — one row per rollout: done / failed

The run I actually executed

Manifest rows	300	
Queries	30 (ids 57 through 99, the train split)	
Seeds	0 through 9	79-s2 died while the planner was
Status done	299	
Status failed	1 (79-s2)	

asking `gpt-5-mini` for the research plan: empty API response, no report. A rerun of the same command would retry only that id, because every other folder already counts as complete.

Counted from `runs/rollouts_drb_d3b4/manifest.jsonl` on disk: 299 done, 1 failed.

What those 300 runs actually contain

Planned rollouts	300	
Finished with a report	299	
Failed (79-s2, empty planner response)	1	The 132 empty runs are queries 81–99, after Report and scraped pages
Report and scraped pages	167	
Report written from zero pages	132	

the Tavily keys hit their cap. They have no evidence pool, so they contribute **zero** BC rows.
Usable slice used for reward and BC: queries **57–80**.

How the corpus died, query by query

Search was the binding constraint, not the orchestrator.

Queries 57–78	10/10 seeds have scraped pages
---------------	--------------------------------

Query 79	9/10 (the miss is 79-s2)
----------	--------------------------

Query 80	8/10
----------	------

Queries 81–99	Reports exist, retained pages = 0
---------------	--

Eight Tavily keys were in rotation.

They exhausted around query 80. After that the agent still called search, got nothing back, and wrote a report from the planner's own text. Those folders look "done" in the manifest. They are not training data: bc-pairs emits 0 rows from them because there is no evidence pool and no features. On the 167 runs that did scrape: mean 26 pages kept, mean 104 reads, mean ~137k tokens, mean ~7 minutes.

What the reward is ranking

On the 166 good rollouts that also have a RACE score:

	mean	correlation with R	
RACE quality Q	0.468 (sd 0.019)	0.10	
Coverage shaping Φ	0.71	0.75	Q barely moves (0.42–0.51).
Lower latency	—	0.62	
Lower tokens	—	0.48	

The reward mostly prefers trajectories that cover the question and finish cheaper. That is a property of this corpus, not a claim that RACE does not matter. BC then drops anything with $Q < 0.44$ so a cheap empty-ish run cannot win on cost alone.

2. The behavior-cloning set

From queries 57–80 only.

Trajectories loaded	170
Kept (top 25% of R , $Q \geq 0.44$, ≥ 2 per query)	52
State–action rows	113
Reward cutoff	0.641 (max 0.699)
Train rows (queries 57–78)	104
Dev rows (queries 79–80, not in the LoRA train file)	9

data/bc/bc_pairs_d3b4_v1_stats.json. Each row is one round: the serialized pool, and the KEEP / ALLOC / DECISION the random policy took.

What one training target looks like

The model is not trained to write the report. It is trained to copy this action.

KEEP: 0609ac4b5505 0aab52b2a926 ...

ALLOC: branchA=0.25 branchB=0.75

DECISION: CONTINUE

Loss is on the assistant turn only. Evidence ids in KEEP have to be copied from the prompt. Inventing an id is a real error, and we count it.

3. LoRA

- Base: Qwen2.5-7B-Instruct, 4-bit. LoRA rank 16 on all projections.
- 3 epochs, lr 2×10^{-4} , batch 1, grad accumulation 8.
- Hardware: A100. Wall time **535 seconds**.

Epoch	Train loss	Val loss	Val token acc
1	0.105	0.126	95.4%
2	0.088	0.109	96.0%
3	0.067	0.107	95.9%

Val loss fell every epoch. Token accuracy is a

soft number: most tokens are ids copied from the prompt.

Dev check: the number that matters

Nine held-out rounds (queries 79–80). Greedy decode.

	First try (max_new_tokens=400)	After raising the cap
Parses as KEEP/ALLOC/DECISION	0.33	1.00
KEEP Jaccard vs. cloned action	0.50	0.61
CONTINUE/TERMINATE accuracy	0.00	0.44
ALLOC L1	0.83	0.62
Invented ids	5	0

The first column was truncation, not a bad adapter. Gold completions run to ~900 tokens because every kept id is spelled out. The adapter we serve is the second column.

4. Fixed rule, the baseline

Same randomized code as the corpus. Four knobs pinned on every val question (see the earlier slide): keep 80%, almost no ranking noise, at least 2 rounds, then always stop.

- It is not the learned model, and it is not the average of the 300 train runs.
- The other seven knobs (feature weights, branch concentration, ...) are still sampled once per question from seed 0. Same question, same weights, if you rerun it.
- All **10/10** val queries finished with a report and real pages.

On query 88 this rule kept 46 pages and pruned 30%, across the two forced rounds. The learned policy kept 37 and stopped after one round.

5. Learned policy online

Same agent, same depth and breadth. The only change is GR_ORCHESTRATOR=learned, calling

the LoRA.	Val queries with a report	8 / 10	Proof
	Orchestrator calls that failed to parse	0 on those 8	
	65	Hit the 30-minute cap. No report.	
	100	Not run. Search was stopped.	

for each finished query is gpt_researcher/<id>.json: parse_failure=false on every round, and the last round's stop reason is model, not a fallback.

Worked example: query 88

Question. How did Netflix adapt *One Hundred Years of Solitude*?

Learned policy, one round, pool of 47 pages. It then stopped, with the whole allocation on one branch, so that split was never used. A round that does split is query 64, round 1 (pool 39, four branches, then CONTINUE):

ALLOC: A=0.11 B=0.05 C=0.41 D=0.43

- On 88 the learned policy dropped 10/47 pages (21%). Fixed rule on the same question dropped 30% and kept 46 pages, but it was forced to run 2 rounds.
- Learned stopped because the model said TERMINATE. `parse_failure` is false.
- On this question the scores match: RACE 0.471 vs. 0.470. FACT support 0.71 vs. 0.75.
- The agent's own citation count was 14 vs. 9. FACT scored 38 vs. 12 claim–URL pairs. Those are different counts.

File: `runs/eval_val_learned/88-s0/gpt_researcher/88.json`.

Pages kept on the 8 shared queries

Same questions. Fixed is the frozen rule. Learned is the LoRA.

id	fixed kept	learned kept	fixed pruned	learned pruned
59	53	78	28%	28%
64	36	64	28%	45%
69	59	71	30%	1%
84	46	34	27%	21%
85	49	97	32%	13%
88	46	37	30%	21%
94	60	95	29%	46%
95	45	92	32%	46%
mean	49	71	30%	28%

Learned keeps more pages on

average because it often runs more rounds before it stops. Prune rate is similar. This is cost and behavior, not quality.

Paired RACE, same 8 questions

Judge: gpt-5-mini on Azure. Official Deep Research Bench RACE. Queries 65 and 100 are

out: learned never finished them.	id	fixed	learned	Δ	Train-corpus RACE on the
	59	0.480	0.471	-0.009	
	64	0.487	0.495	+0.008	
	69	0.468	0.467	-0.001	
	84	0.430	0.446	+0.016	
	85	0.485	0.482	-0.003	
	88	0.470	0.471	+0.001	
	94	0.463	0.483	+0.020	
	95	0.490	0.483	-0.007	
	mean	0.471	0.475	+0.003	

166 rollouts that kept pages was 0.468, sd 0.019. A gain of 0.003 is inside that noise. Breakdown, fixed $\rightarrow$ learned: comprehensiveness +0.005, insight +0.003, instruction following +0.001, readability +0.006.

Paired FACT, public reader

Same reports. The paid Jina key returned 402, so every cited URL was fetched by putting `https://r.jina.ai/` in front of it, with no API key. The judge is still gpt-5-mini.

id	fixed support	learned support	fixed claims	learned claims
59	0.80	0.82	15	22
64	0.45	0.39	20	23
69	0.96	0.94	24	18
84	0.63	0.63	19	16
85	0.78	0.46	46	37
88	0.75	0.71	12	38
94	0.75	0.59	36	34
95	0.60	0.58	10	12
mean	0.72	0.64	23	25

Support rate is the fraction

of extracted claim–URL pairs the judge marked supported. Learned is lower on 7 of 8 questions. Queries 85 and 94 are most of the gap.

What that quality cost

Means on the same 8 questions.		fixed rule	learned	Learned keeps more
	Pages kept	49	71	
	Pages read	96	143	
	Prune rate	30%	28%	
	Tokens	139k	224k	
	Wall clock	6.9 min	14.7 min	

because it often runs past the fixed rule's forced stop at round 2. The extra search did not raise RACE, and the share of claims that check out went down.

What I will not claim in the meeting

- Not “the learned policy beats the fixed rule.” RACE moved by $+0.003$. FACT support went from 0.72 to 0.64, and the run cost about twice as much.
- Not a test-set number. Queries 51–55 have not been touched.
- Not that token accuracy 96% means the decisions match. Jaccard is 0.61 and the stop bit is right 44% of the time on 9 dev rounds.
- Not that the 132 empty train rollouts are data. They were excluded.
- Not that the first FACT pass counted citations. That pass was a Jina 402. The numbers above are the public-reader rescore.

What is solid

The pipeline is real, and on these 8 questions the LoRA does not improve the report. It copies a legal action, searches longer, and lands on the same RACE score.

Where the proof lives

Claim	File
Randomized policy code	orchestration/randomized.py
Rollout driver	src/adr/rollouts/driver.py
299 done / 1 failed	runs/rollouts_drb_d3b4/manifest.jsonl
Splits, test held out	data/benchmarks/.../splits.json
52 trajectories, 113 rows	data/bc/bc_pairs_d3b4_v1_stats.json
Train 104 / dev 9	data/bc/oai_train.jsonl, oai_dev.jsonl
LoRA losses	Colab log, epoch table on the training slide
Dev Jaccard / 0 hallucinated ids	Section 3 of pilot_bc_qwen_lora.ipynb
Query 88 action	runs/eval_val_learned/88-s0/gpt_researcher/88.json
Kept-page table	metrics/summary.json under each *-s0
Paired RACE and FACT	runs/eval_val_compare.md

- 1 The val comparison is done at $n = 8$. Leave 65 and 100 out, or rerun 65 with a longer cap and then 100.
- 2 Do not open the test split until the val policy is frozen. Nothing here says the learned policy should replace the fixed rule.
- 3 If FACT is quoted again, say it used the public `r.jina.ai` reader. The paid key has no balance.